الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Lecture 5: Handout

Introduction to Assembly Language Programming

C – Programming language Basics

1. Comments in C Language

- Single-line comments: `//`
- Multi-line comments:

```c
/*
Comment 1
Comment 2
*/
```

2. Constants and Libraries

- Define constants:

```c
#define PI 3.1415  // Defining the constant PI
```

- Include header files:

```c
#include <stdio.h>   // Standard input-output header
#include <math.h>    // Math library for functions like sqrt, pow
```

3. Example Program Structure

```c
int main() {
    int x;
    double y;
    float z;

    x = factorial(4);  // Calculate factorial of 4
    x = 10;            // Assign 10 to x
    y = sqrt(x);       // Square root of x
    z = pow(z, 2);     // z squared

    scanf();   // Input function placeholder
    printf();  // Output function placeholder

    return 0;  // Program exit
}
```

- `return 0;` signifies successful program termination.

4. Factorial Function Example

- Definition:

```c
int factorial(int x) {
    // Code for factorial calculation
}
```

Assembly Language Basics

1. Program Layout

- Assembly language programs include sections for data definition, code, and additional settings.

2. Data Types in Assembly

- Examples:

```asm
DATA SEGMENT
    ; Byte-sized data (8-bit)
    DATA_BYTE DB 42         ; Decimal value
    DATA_CHAR DB 'A'        ; Character 'A'
    DATA_HEX DB 2AH         ; Hexadecimal value 2A (42 in decimal)

    ; Word-sized data (16-bit)
    DATA_WORD DW 1234H      ; Hexadecimal value
    DATA_INT DW 256         ; Decimal value (256)

    ; Double Word-sized data (32-bit)
    DATA_DWORD DD 12345678H ; Hexadecimal value (32-bit)
    DATA_LONG DD 123456789  ; Large decimal value (32-bit)

    ; Quad Word-sized data (64-bit)
    DATA_QWORD DQ 123456789ABCDEF0H ; 64-bit hexadecimal value
```

```c
char x = 'A'
int x;
double x;
```

3. Register Manipulation and System Interrupts

- Example of a software interrupt:

```asm
mov ax, 4ch    ; Load value 4Ch into AX register
int 21h        ; Call DOS interrupt to terminate program
```

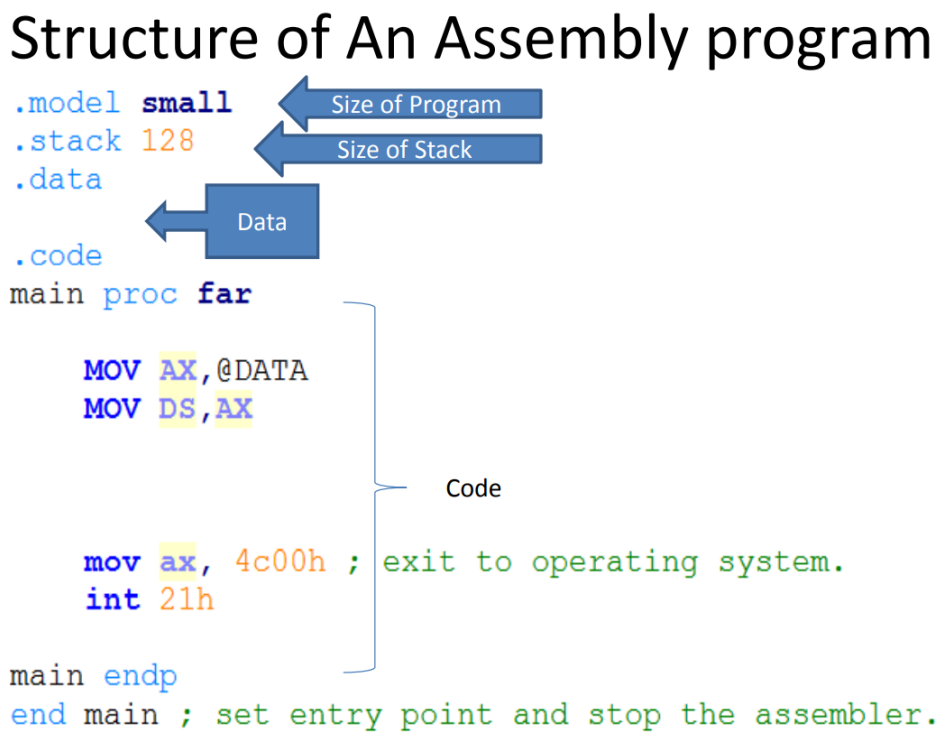
```c
sqrt(x);
pow(z, 2);

scanf();
printf();
```

4. Basic Syntax and Error Handling

- Correct data segment load:

```asm
mov ax, @data    ; Load data segment address
mov ds, ax       ; Move data segment address into DS register
```

- **Note**: Direct movement to the segment register (`mov ds, 1234h`) is invalid.



5. Sample Code - Addition of Two Numbers

```asm
; Define two numbers
DATA1 db 52H    ; 8-bit value in hexadecimal
DATA2 db 83     ; Decimal value

; Adding two numbers example
mov al, DATA1
add al, DATA2   ; AL = DATA1 + DATA2
```

Sample Assembly Code: Adding Two Numbers (DATA1 and DATA2)

```
        .MODEL SMALL
        .STACK 64
        .DATA
DATA1   DB 52h
DATA2   DB 29h
SUM     DB ?
        .CODE
MAIN    PROC FAR
        MOV AX,@DATA
        MOV DS,AX
        MOV AL,DATA1
        MOV BL,DATA2
        ADD AL,BL
        MOV SUM,AL
        MOV AH,4Ch
        INT 21h
MAIN    ENDP
END MAIN
```

6. Procedure Definitions

- **Near Procedure** (`PROC NEAR`): For procedures within 256 bytes.
- **Far Procedure** (`PROC FAR`): For larger or external procedures.

7. Data Declaration Examples

- Declare data with different formats:

```asm
DATA1 DB 19H      ; Hexadecimal
DATA2 DB 25       ; Decimal
```

8. Common Assembly Instructions

- `MOV` for data transfer:

```asm
mov [ax], [bx]   ; Error - Direct memory-to-memory transfer is invalid.
mov ds, ax       ; Valid syntax
```

9. Arithmetic Instructions

- **Addition**: `ADD AX, BX` - Adds BX to AX.
- **Increment/Decrement**:

```asm
INC AX   ; Equivalent to AX = AX + 1
DEC BX   ; Equivalent to BX = BX - 1
```

10. Copying Arrays Using Assembly

```asm
mov si, offset Array1   ; Load base address of Array1 into SI
mov di, offset Array2   ; Load base address of Array2 into DI
```

- **Example**: Copying data from Array1 to Array2.

Example Program - Adding Two Long Numbers

- **Task**: Add two 8-byte (64-bit) numbers.

```asm
DATA1 DQ 548F9963CE7h     ; Allocate 8 bytes for DATA1
DATA2 DQ 3FCD4FA23B8Dh    ; Allocate 8 bytes for DATA2
DATA3 DQ ?                ; Allocate 8 bytes for result

mov si, offset DATA1
mov di, offset DATA2
mov bx, offset DATA3
mov cx, 4                ; Loop 4 times (for each 16-bit segment)

BACK:
    mov ax, [si]         ; Load 16-bit segment of DATA1
    add ax, [di]         ; Add corresponding 16-bit segment of DATA2
    mov [bx], ax         ; Store result in DATA3
    add si, 2            ; Move to next 16-bit segment
    add di, 2
    add bx, 2
    loop BACK            ; Repeat until done
```

Assignment

- Write an assembly program using TASM emulator to add two 32-bit numbers.